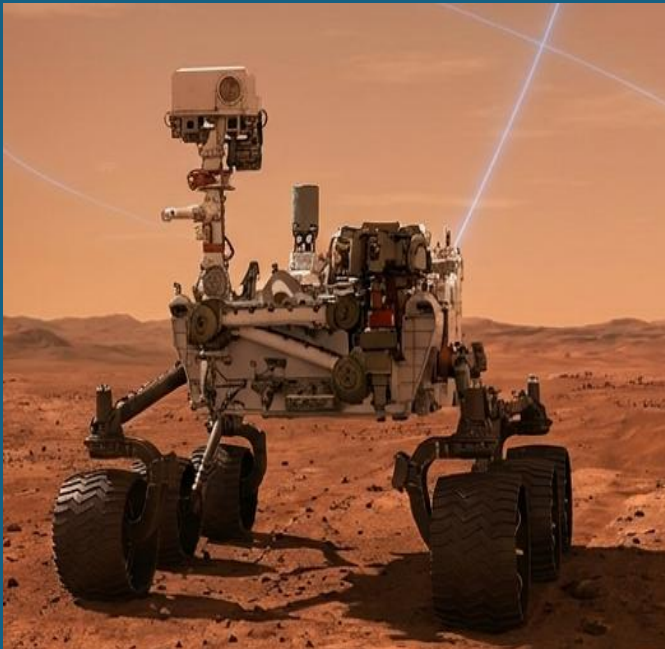


UADE

FUNDAMENTOS DE MODELADO Y SIMULACIÓN



Omar J. Caceres

DEPARTAMENTO DE CIENCIAS BÁSICAS

Segunda Edición 2026

Índice

	Pg.
Introducción	4
Capítulo I	5
Búsqueda binaria de raíces	8
Método del punto fijo	11
Método de aceleración Aitken	13
Método de Newton Raphson	15
Polinomio Interpolante de Lagrange	20
Diferencias finitas	24
Reglas de Newton Cotes	27
Regla del Rectángulo medio compuesto	27
Regla del trapecio	27
Regla de Simpson	28
El Método de Monte Carlo	36
Ecuaciones diferenciales	44
Método de Euler	46
Método de Euler mejorado (Heun)	48
Método de Runge Kutta 4	50
Bibliografías Capítulo I	57
Capítulo II Sistemas dinámicos	58
Conceptos Básicos	61
Sistemas desacoplados unidimensionalmente	63
Bifurcaciones	71
Tipo Hopf	75
Sistema Lorent	78
Sistemas dinámicos lineales 2d	79
Sistemas lineales no homogéneos	91
Conversión de un sistema (EDO) de orden superior	94
Sistemas dinámicos no lineales	95
Análisis cualitativo	98
Teoremas de existencia y unicidad (Picard -Lindelöf)	98
Teorema de Harmant-Grobman	98
Estabilidad de Lyapunuv (directa)	99
Principio de Invarianza deSalle	100
Teorema del flujo	101
Teorema de la variedad estable e inestable	104
Teorema de la variedad central	104
Teorema de Poincaré Bendinxson	104
Teorema de Bendixson-Dulac	107
Teorema de Invarianza positiva (criterio de la frontera)	110
Teorema de la Bifurcación Hopf	110

Teorema de la estabilidad estructural	110
Bibliografías Capítulo II	139
Aplicaciones de sistemas dinámicos	148
Romeo y Julieta	148
Modelo Lotka Volterra	155
Modelos de combate	158
Sistemas conservativos	164
Parametrización de sistemas dinámicos	168
Estructura Orbital	172
Conjuntos límites	173
Glosario de términos básicos	195
Prácticas de Laboratorio	201
Bibliografía Modelos	211

Introducción

Modelado y simulación es sin dudas una de las materias más prácticas y vigentes en el campo de la ciencia y la ingeniería. Sin embargo, no es una disciplina nueva ya que desde tiempos remotos la humanidad tuvo la necesidad de describir fenómenos naturales —como el movimiento planetario, los brotes epidémicos o los cambios en la economía, el comportamiento del clima, o cómo funcionan los biorritmos humanos, esto ha impulsado el desarrollo de modelos que permitan comprender, anticipar y, en cierta medida, controlar el comportamiento de los procesos.

Las primeras civilizaciones trazaron el cielo para predecir las estaciones; más tarde, médicos y filósofos intentaron explicar la propagación de enfermedades; y con el surgimiento de la economía moderna tratar de entender los cambios en los mercados, por esto matemáticos y economistas buscaron patrones que revelaran la evolución de la dinámica comercial.

A lo largo de la historia descubrimos que detrás de la diversidad de fenómenos existen estructuras regulares que pueden expresarse mediante ecuaciones, algoritmos o reglas lógicas matemáticas. Así nacieron los primeros modelos: simplificaciones del mundo real que capturan su esencia y permiten razonar sobre ella. En la era moderna, el modelado y la simulación por computadora se volvieron herramientas indispensables. La física formalizó leyes que describen desde osciladores hasta sistemas planetarios; la biología desarrolló modelos poblacionales y epidemiológicos; la ingeniería utiliza representaciones matemáticas para diseñar estructuras seguras; la computación permite simular sistemas en escenarios imposibles de experimentar directamente.

Sin embargo, todo modelo —por simple o complejo que sea— necesita ser resuelto. Y para resolverlo, especialmente cuando la solución analítica no existe o es impracticable, recurrimos a los métodos numéricos. En este texto se presentan de una manera resumida y sencilla estos métodos, al menos los más comunes, siendo la primera parte los métodos numéricos mientras que en la parte final tendremos los sistemas dinámicos.

CAPÍTULO I MÉTODOS NUMÉRICOS

Los métodos numéricos son un conjunto de técnicas y algoritmos diseñados para aproximar soluciones a problemas matemáticos que no pueden resolverse de forma exacta o analítica, o cuya solución exacta resulta impráctica y compleja de calcular. Estos métodos se apoyan en procedimientos iterativos, y el uso de aproximaciones sucesivas donde el cálculo computacional es primordial, por lo que son fundamentales en ciencias aplicadas e ingeniería.

Conceptos básicos:

a) Modelo Matemático “Se define, de manera general, como una formulación o una ecuación que expresa las características esenciales de un sistema físico o de un proceso en términos matemáticos. en general, el modelo se representa mediante una relación funcional de la forma: Variable dependiente = f (variables independientes, parámetros, funciones de fuerza)” Steven Chapra Métodos Numéricos para ingenieros 5ta Edición, Pg 11.

b) Simular: una vez que se ha creado un modelo, la simulación es el proceso de ejecutarlo para estudiar el comportamiento del sistema a través del tiempo o bajo diferentes condiciones.

c) Iterar: Significa repetir un proceso con el objetivo de acercarse a un resultado deseado.

$$x_{n+1} = x_n$$

d) Convergencia: Indica como las aproximaciones sucesivas se acercan a la solución x^* de una ecuación: $f(x) = 0$

Velocidad de convergencia: La rapidez con la que se reduce el error, $e_n = |x_n - x^*|$

e) Orden de Convergencia: Es una medida utilizada en análisis numérico para describir la rapidez con la que una secuencia iterativa se acerca a su límite o solución. cuando se emplean métodos iterativos para resolver ecuaciones o sistemas de ecuaciones, es útil conocer el orden de convergencia para determinar la eficiencia del método.

$$\lim_{n \rightarrow \infty} \frac{|X_{n+1} - X^*|}{|X_n - X^*|^P} = C$$

Donde C es una constante, P el orden de aproximación y X^* la solución buscada, es decir se basa en cómo se reduce el error entre aproximaciones sucesivas. Entonces:

$$|e_{n+1}| = C|e_n|^p$$

Donde C es una constante independiente de n, P el orden de convergencia, X^* la solución buscada, e_n error.

$$e_n = x_n - x^*, C > 0, p > 1$$

Algunos Métodos Iterativos: Existen otros métodos, pero estos son los más conocidos.

Método	Orden de Convergencia	Ventajas	Desventajas
Newton Raphson	Cuadrática	Muy rápido cerca del valor real	Necesita derivadas
Bisección	Lineal	Siempre converge	Lento
Punto Fijo	Lineal	Fácil de implementar	Requiere la condición $ g'_{(x_0)} < 1$

TABLA 1

Cota de error: La cota de error se refiere a una estimación del error máximo que se comete en un proceso iterativo o aproximación. Garantiza el resultado dentro de una tolerancia aceptable.

Tipos de cota de error: existen varios tipos, acá estudiaremos los dos principales que son; error absoluto y relativo.

Error Absoluto: Es la diferencia entre el valor exacto x^* y la aproximación x_n

$$E_{abs} = |x^* - x_n|$$

Como x^* no lo conocemos entonces se puede usar diferencia entre iteraciones

$$E_{abs} \approx |x_{n+1} - x_n|$$

Error Relativo: Es una medida utilizada para cuantificar la precisión de una aproximación o medición en comparación con un valor exacto o verdadero. se expresa como una fracción o porcentaje del valor verdadero y es útil para evaluar la significancia del error en términos relativos.

$$E_r = \frac{|x^* - x_n|}{|x^*|}$$

Como x^* no lo conocemos entonces se puede usar diferencia entre iteraciones

$$E_r \approx \frac{|x_{n+1} - x_n|}{|x_{n+1}|}$$

Criterios de Detención: Son las reglas que definen cuando un método iterativo debe parar, los criterios más comunes son:

1. Tolerancia en el error absoluto: $|x_{n+1} - x_n| \leq \varepsilon$, ε Tolerancia permitida

2. Tolerancia en el error relativo: $\frac{|x_{n+1}-x_n|}{|x_{n+1}|} \leq \varepsilon$
3. Condición sobre el residuo: $|f(x)| \leq \varepsilon$
4. Número máximo de iteraciones: $n > N_{max}$

Como x^* no lo conocemos entonces se puede usar diferencia entre iteraciones

Conjuntos compactos:

En un espacio métrico, un conjunto k se define compacto si es cerrado (contiene todos los puntos límite), acotado: (todos los puntos del conjunto están a una distancia finita de algún punto fijo).

Ejemplos de conjuntos compactos:

1. Un disco cerrado en el plano $\{(x, y) \in \mathbb{R}^2, x^2 + y^2 \leq 1\}$

es compacto, es un disco de radio 1 centro en el origen, es cerrado porque incluye el borde $x^2 + y^2 = 1$ y es acotado

Ejemplos de conjuntos compactos:

2. Un segmento de recta en $\mathbb{R}^n$: la línea parametrizada $[a, b] = \{X \in \mathbb{R}^n, x = ta + (1 - t)b, 0 \leq t \leq 1\}$ es compacta.
3. Un rectángulo cerrado en $\mathbb{R}^2$ el conjunto $[a, b] \times [c, d]$ en $\mathbb{R}^2$ es compacto en la región rectangular delimitada por a, b, c, d (cerrado y acotado).
4. El conjunto de funciones continuas acotadas en $[0, 1]$ con la topología de la convergencia uniforme, esta es una aplicación del teorema de Arzelá-Ascoli, que afirma que un conjunto es compacto si es equicontinuo, acotado y cerrado.

Condición de Lipschitz en conjuntos compactos:

Supongamos que tenemos una función $g: k \rightarrow \mathbb{R}$, definida en un conjunto compacto k . Decimos que g satisface una condición de Lipschitz constante L , si para todos $x, y \in k$; $|g(x) - g(y)| \leq L|x - y|$.

Si una función es continuamente diferenciable, ($g \in C^1(k)$) entonces g y el conjunto compacto tiene una derivada acotada, $|g'(x)| < M$, para todo $x \in k$ y podemos tomar $L=M$.

La constante L es uniforme en el todo el conjunto compacto, y además da garantía de convergencia si $L < 1$ en $x_{n+1} = g(x_n)$.

Propiedades de un conjunto compacto:

Existencia de un punto fijo: El teorema del punto fijo de Banach establece que, si X es un conjunto compacto completo y compacto en un espacio métrico, y f es una función contractiva, entonces existe un único punto fijo. $x^* \in X$ tal que $f(x^*) = x^*$

Convergencia Iterativa: Partiendo de un punto arbitrario $x_0 \in X$ la sucesión generada la iteración de f : converge al punto fijo, $x_{n+1} = f(x_n)$

Funciones Contractivas: sea una $f: X \rightarrow X$, Se dice contractiva si existe una constante $k \in [0,1)$ tal que $d(f(x), f(y)) \leq kd(x, y)$, $\forall x, y \in X$, donde $d(\cdot, \cdot)$ es una métrica definida en el espacio métrico X .

En esencia una función contractiva (acercas) los puntos del dominio, ya que la distancia entre las imágenes de los puntos es estrictamente menor que la distancia entre los puntos originales

Ejemplo Función contractiva: sea $f(x) = \frac{1}{2}x$, definida en un conjunto compacto $X = [0,1]$.

Verificamos la contracción: $|f(x) - f(y)| = \left| \frac{1}{2}x - \frac{1}{2}y \right| = \frac{1}{2}|x - y|$

En este caso la constante de contracción cumple el requerimiento, entonces, hay un único punto fijo en X . se calcula $f(x) = x$, ordenando la ecuación: $\frac{1}{2}x - x = 0$, por lo que $x^* = 0$, esto nos dice que, si aplicamos iteración desde un punto cercano por ejemplo 1, la función converge a 0.

$$d(x_n, x^*) \leq k^n d(x_0, x^*), |f(x) - f(y)| \leq k|x - y|, 0 < k < 1$$

Búsqueda binaria de raíces

También conocida como el **método de bisección**, es un algoritmo numérico utilizado para encontrar una raíz de una función continua $f(x)$ en un intervalo dado $[a, b]$, donde $f(a)$ y $f(b)$ tienen signos opuestos $f(a) \cdot f(b) < 0$. este método se basa en el teorema del valor intermedio, que garantiza la existencia de al menos una raíz en dicho intervalo.

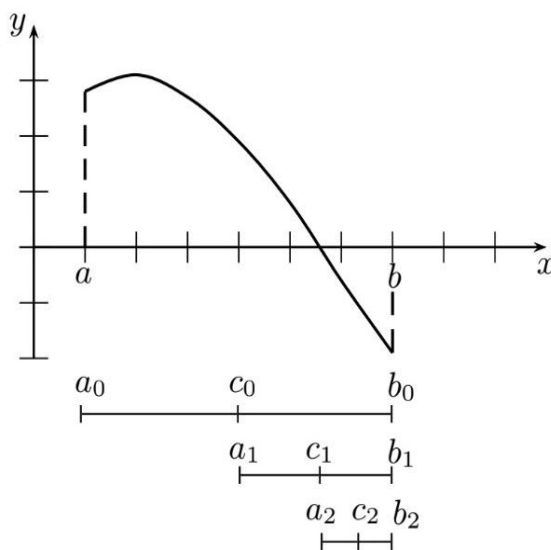


FIGURA 1.

Teorema de Bolzano

sea $f(x)$ una función continua en un intervalo cerrado $[a, b]$. si $f(a) \cdot f(b) < 0$, es decir, $f(a)$ y $f(b)$ tienen signos opuestos, entonces existe al menos un punto $c \in (a, b)$ tal que: $f(c) = 0$

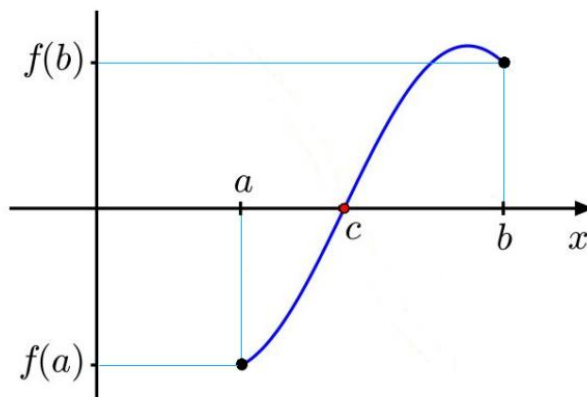


FIGURA 2

El algoritmo consiste en seguir los siguientes pasos:

1. Escoger el intervalo $[a, b]$ donde $f(a) \cdot f(b) < 0$, esto garantiza que haya una raíz en dicho intervalo.
2. Calcular el punto medio. $c = \frac{a+b}{2}$
3. Evaluar c en la función: si $f(c) = 0$, entonces parar y hemos obtenido la raíz.
4. Si $f(a) \cdot f(c) < 0$, entonces la raíz está en $[a, c]$ y actualizamos $b = c$
5. Si $f(b) \cdot f(c) < 0$, entonces la raíz está en $[c, b]$ actualizamos $a = c$

Código en python

```
import numpy as np
import matplotlib.pyplot as plt
from tabulate import tabulate
# Método de Bisección
def biseccion(f, a, b, iteraciones=100, tolerancia=1e-6, precision=5):
    if f(a) * f(b) >= 0:
        raise ValueError("La función debe tener signos opuestos en los extremos del intervalo [a, b].")
    results = []
    for i in range(iteraciones):
        c = round((a + b) / 2.0, precision)
        fc = round(f(c), precision)
        results.append([i+1, a, b, c, fc])
        print(tabulate(results, headers=["Iteración", "a", "b", "c", "f(c)"], tablefmt="grid"))
        if abs(fc) < tolerancia or (b - a) / 2.0 < tolerancia:
            return c
```

```

    if f(a) * f(c) < 0:
        b = c
    else:
        a = c
    raise ValueError("El método no convergió o faltan iteraciones.")
def graficar_biseccion(f, a, b, raiz):
    # Graficar la función
    x = np.linspace(a - 1, b + 1, 400)
    y = f(x)
    plt.plot(x, y, label='$f(x)$')
    plt.axhline(0, color='black', linewidth=0.5)
    plt.axvline(0, color='black', linewidth=0.5)
    plt.grid(color='gray', linestyle='--', linewidth=0.5)
import numpy as np
import matplotlib.pyplot as plt
from tabulate import tabulate
# Método de Bisección
def biseccion(f, a, b, iteraciones=100, tolerancia=1e-6, precision=5):
    if f(a) * f(b) >= 0:
        raise ValueError("La función debe tener signos opuestos en los extremos del intervalo [a, b].")
    results = []
    for i in range(iteraciones):
        c = round((a + b) / 2.0, precision)
        fc = round(f(c), precision)
        results.append([i+1, a, b, c, fc])
        print(tabulate(results, headers=["Iteración", "a", "b", "c", "f(c)"], tablefmt="grid"))
        if abs(fc) < tolerancia or (b - a) / 2.0 < tolerancia:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    raise ValueError("El método no convergió o faltan iteraciones.")
def graficar_biseccion(f, a, b, raiz):
    # Graficar la función
    x = np.linspace(a - 1, b + 1, 400)
    y = f(x)
    plt.plot(x, y, label='$f(x)$')
    plt.axhline(0, color='black', linewidth=0.5)
    plt.axvline(0, color='black', linewidth=0.5)
    plt.grid(color='gray', linestyle='--', linewidth=0.5)

```

Método del punto fijo

Es un punto donde al evaluarlo en una función se obtiene el mismo punto

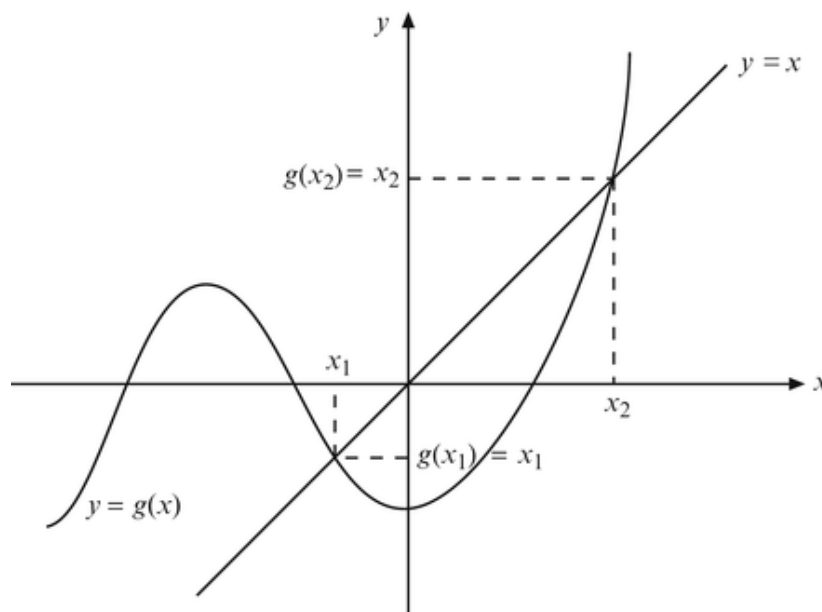


FIGURA 3

Propiedades de un conjunto compacto:

Existencia de un punto fijo: El teorema del punto fijo de Banach establece que, si X es un conjunto compacto completo y f es una función contractiva, entonces existe un único punto fijo. $x^* \in X$ tal que $f(x^*) = x^*$

converge a x^* :

$$\lim_{n \rightarrow \infty} x_n = x^*$$

Convergencia Iterativa: Partiendo de un punto arbitrario $x_0 \in X$ la sucesión generada la iteración de f : converge al punto fijo, $x_{n+1} = f(x_n)$

Sea $f_{(x)}$ la función original,
 $g_{(x)}$ la función a iterar

Para aplicar Banach:

- $g: X \rightarrow X$ debe ser contractiva
- X debe ser completo

Entonces el punto fijo de g es la solución buscada.

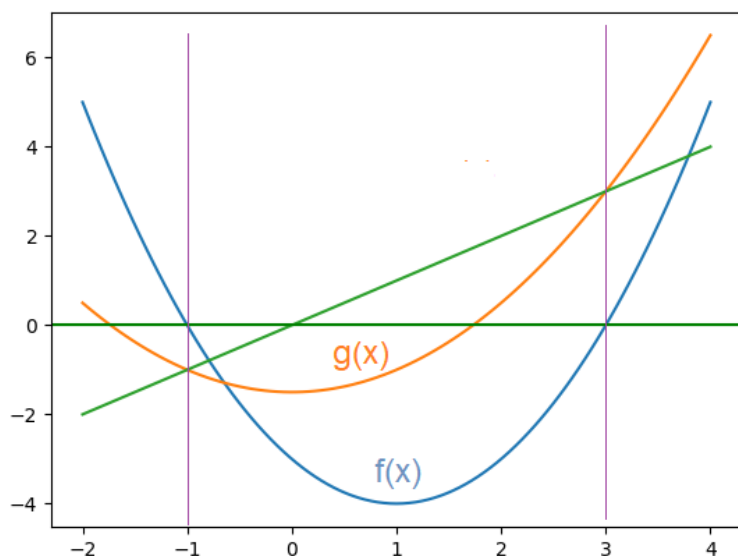


FIGURA 4

Casos

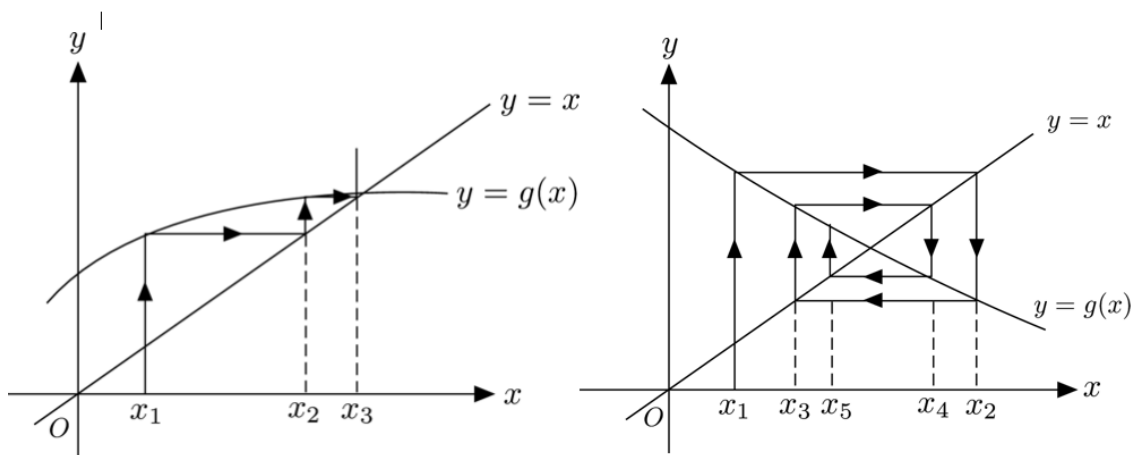


FIGURA 5

Código en Python

```
import math
import numpy as np
import matplotlib.pyplot as plt
"""
f(x)=cos(x)
0=cos(x)
x= cos(x)+x
"""
```

```
def f(x):
    return math.cos(x)
def g(x):
    return math.cos(x) + x
def fixed_point_iteration(x0, tol=1e-5, max_iter=100):
    x = x0
    iter_values = [x0]
    for i in range(max_iter):
        x_new = g(x)
        iter_values.append(x_new)
        if abs(x_new - x) < tol:
            print("Tolerance exceeded...")
            break
        x = x_new
    return x_new, iter_values
x0 = 1.0 # Valor inicial
root, iter_values = fixed_point_iteration(x0)
# Graficar la función original y el proceso de iteración
x_vals = np.linspace(-2 * np.pi, 2 * np.pi, 400)
y_vals = np.cos(x_vals)

plt.plot(x_vals, y_vals, label='$f(x) = \cos(x)$')
plt.scatter(iter_values, [f(x) for x in iter_values], color='red', zorder=5)
plt.plot(iter_values, [f(x) for x in iter_values], color='red', linestyle='--', zorder=5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(color='gray', linestyle='--', linewidth=0.5)
plt.title('Método del Punto Fijo para $f(x) = \cos(x)$')
plt.xlabel('$x$')
plt.ylabel('$f(x)$')
plt.legend()
plt.show()

print(f"La raíz aproximada es: {root}")
```

Método de aceleración Aitken

Es una técnica numérica que se usa para mejorar la velocidad de convergencia de una secuencia que converge lentamente hacia un límite. básicamente, toma tres elementos consecutivos de una secuencia y calcula un valor "acelerado" que se acerca más rápidamente al valor límite. la fórmula central del método es:

$$x_n^* = x_{(n)} - \frac{(x_{n+1} - x_n)^2}{x_{n+2} - 2(x_{n+1}) + x_n}$$

$x_{(n)}$ termino de sucesión original

x_n^* termino de sucesión acelerado

Se requiere tres términos iterativos consecutivos para poder iniciar la primera extrapolación, y este valor puede ser considerado el inicial de un nuevo proceso de aceleración tras calcular los próximos dos términos al mismo

Condiciones de Convergencia

- Secuencia Convergente: La secuencia debe ser monótona y convergente hacia un valor límite L. El método no es efectivo si la secuencia diverge o si oscila de manera significativa.
- Disponibilidad de Tres: El denominador de la fórmula de Aitken.
- Términos Consecutivos: Para aplicar la fórmula de Aitken, necesitas al menos tres términos consecutivos de la secuencia.
- Diferencia no Nula en el Denominador.
- Convergencia Relativamente Lenta: El método es especialmente útil cuando la secuencia converge lentamente al límite, ya que Aitken acelera el proceso. Si la secuencia ya converge rápidamente, la mejora puede ser mínima.
- Estabilidad Numérica: Si la secuencia presenta un comportamiento numérico inestable o errores significativos en los cálculos, los resultados de la aceleración pueden no ser fiables.

Código

```
def punto_fijo_con_aitken_tabla(g, x0, tol=1e-6, max_iter=100):
    """
    Método del punto fijo con aceleración de Aitken que muestra una tabla de iteraciones.
    g: función de iteración (polinomio en este caso).
    x0: aproximación inicial.
    tol: tolerancia para la convergencia.
    max_iter: número máximo de iteraciones.
    """
    x = x0
    print(f'Iteración':<10f'x':<20f'x1 = g(x)':<20f'x2 = g(x1)':<20f'x_acelerado':<20f'Error':<20f')
    print("-" * 100)
    for i in range(max_iter):
        # Calcular tres puntos consecutivos de la iteración
        x1 = g(x)
        x2 = g(x1)
        # Aplicar la aceleración de Aitken
        denominador = x2 - 2 * x1 + x
        if denominador != 0:
            x_acelerado = x - (x1 - x)**2 / denominador
        else:
            x_acelerado = x2 # Si no se puede aplicar Aitken, continuar con x
        # Calcular error relativo
```

```

error = abs(x_acelerado - x)
# Mostrar valores en la tabla
print(f"{i + 1:<10}{x:<20.10f}{x1:<20.10f}{x2:<20.10f}{x_acelerado:<20.10f}{error:<20.10f}")
# Verificar convergencia
if error < tol:
    print(f"\nConvergencia alcanzada en la iteración {i + 1}.")
    return x_acelerado
# Actualizar para la siguiente iteración
x = x_acelerado
print("\nEl método no converge después del número máximo de iteraciones.")
return None
# Ejemplo de uso
def g(x):
    return (2*x-1)**(1/2) # Polinomio g(x)
x0 = 2 # Aproximación inicial
resultado = punto_fijo_con_aitken_tabla(g, x0)
if resultado is not None:
    print(f"\nLa solución aproximada es: {resultado}")

```

Método de Newton Raphson

El método de Newton-Raphson es un método iterativo muy eficiente para encontrar raíces de funciones no lineales $f(x)=0$. se basa en la aproximación lineal de la función mediante su derivada, y su fórmula iterativa es:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

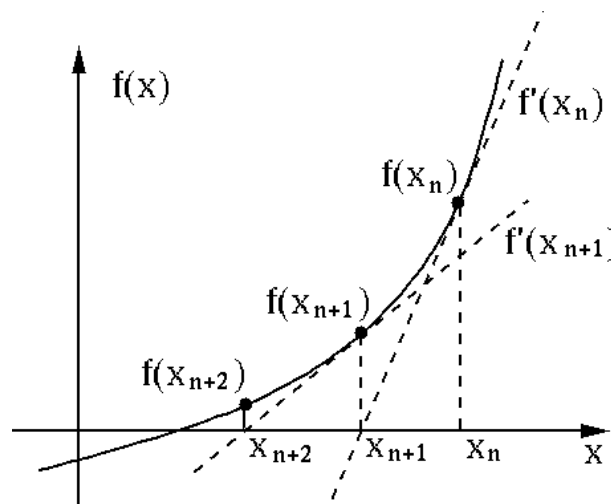


FIGURA 6

Ventajas:

- Convergencia rápida si el punto inicial está cerca de la raíz.
- Simple de implementar.

Limitaciones:

- Puede fallar o divergir si la derivada es cero o si el punto inicial no está cerca de la raíz.
- Requiere conocer la derivada de la función.

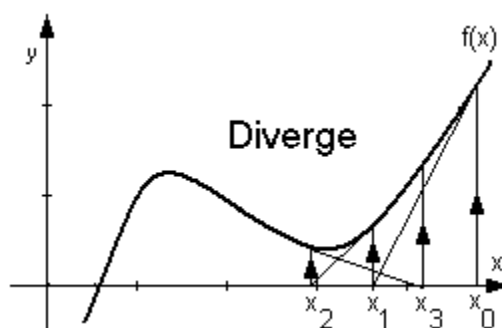


FIGURA 7

Código

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.misc import derivative
from tabulate import tabulate
# Método de Newton-Raphson
def newton_raphson(f, valor_inicial, iteraciones=100, tolerancia=1e-6, precision=5):
    x = valor_inicial
    results = []
    for i in range(iteraciones):
        fx = round(f(x), precision)
        dfx = round(derivative(f, x, dx=tolerancia), precision)
        if dfx == 0:
            raise ValueError("La derivada es cero. El método no puede continuar.")
        x_new = round(x - fx / dfx, precision)
        results.append([i+1, x, fx, dfx, x_new])
        print(tabulate(results, headers=["Iteración", "x", "f(x)", "f'(x)", "Resultado"], tablefmt="grid"))
        if abs(x_new - x) < tolerancia:
            return x_new
        x = x_new
    raise ValueError("El método no convergió o faltan iteraciones.")
def graficar(f, raiz):
```



```
# Graficar la función
x = np.linspace(0, 3, 400)
y = f(x)
plt.plot(x, y, label='$f(x)$')
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(color = 'gray', linestyle = '--', linewidth = 0.5)
# Marcar la raíz encontrada
plt.plot(raiz, f(raiz), 'ro', label=f'Raíz: x = {raiz:.5f}')
# Añadir leyenda y mostrar la gráfica
plt.legend()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Gráfica de la función y su raíz')
plt.show()
# Definir la función para la cual quieres encontrar la raíz
def f(x):
    return x**3 - x - 4
# Valor inicial
valor_inicial = 1.0
# Encontrar la raíz utilizando el método de Newton-Raphson
raiz = newton_raphson(f, valor_inicial)
# Imprimir la raíz encontrada
print(f"La raíz encontrada es: {raiz}")
# Graficar la función y la raíz
graficar(f, raiz)
```

Ejercicios búsqueda de raíces

Búsqueda Binaria de raíces (Bisección)

1. Para cada una de las funciones halle un intervalo $[a, b]$, de manera que $f(a)$ y $f(b)$ tengan signo contrario.

- a) $f(x) = e^x - 2 - x$
- b) $f(x) = \cos(x) + x$
- c) $f(x) = \ln(x) - 5 - x$
- d) $f(x) = x^2 - 10x + 23$

2. Sea $f(x) = 3(x + 1) \left(x - \frac{1}{2}\right) (x - 1)$ aplique el método de búsqueda binaria de raíces en los siguientes intervalos:

- a) $[-1, 1.5]$
- b) $[-1.25, 2.5]$

3. Aplique el método de bisección para encontrar una solución aproximada con tolerancia de 10^{-3} , para las siguientes funciones en sus intervalos:

- $\sqrt{x} - \cos(x) = 0$, para $0 \leq x \leq 1$
- $x - 2^{-x} = 0$, para $0 \leq x \leq 1$
- $e^x - x^2 + 3x - 2 = 0$, para $0 \leq x \leq 1$
- $2x \cos(x) - (x + 1)^2 = 0$, para $-3 \leq x \leq -2$, para $-1 \leq x \leq 0$
- $x \cos(x) - 2x^2 + 3x - 1 = 0$, para $0.2 \leq x \leq 0.3$, para $1.2 \leq x \leq 1.3$

4. Aplique el método de bisección para encontrar todas las raíces del polinomio dentro de 10^{-2} , para $x^4 - 2x^3 - 4x^2 + 4x + 4$, en:

- $[-2, -1]$
- $[0, 2]$
- $[2, 3]$
- $[-1, 0]$

5 sea $f(x) = (x + 2)(x + 1)(x - 1)^3(x - 2)$, ¿a cuál cero la función converge, estudie los siguientes intervalos?

- $[-3, 2.5]$
- $[-2.5, 3]$
- $[-1.75, 1.5]$
- $[-1.5, 1.75]$

Use el método del punto fijo para

- $f(x) = 2e^{x^2} - 5x$, $x^* \in [0, 1]$, $x_0 = 0$
- $f(x) = \cos(x)$, $x^* \in [1, 2]$, $x_0 = 1$
- $f(x) = e^{-x} - x$, $x^* \in [0, 1]$, $x_0 = 0$
- $f(x) = x^3 - x - 1$, $x^* \in [1, 2]$, $x_0 = 1$
- $f(x) = \pi + 0.5 \sin\left(\frac{x}{2}\right) - x$, $x^* \in [0, 2\pi]$, $x_0 = 0$

6. Use manejo algebraico para demostrar que las siguientes funciones tienen un punto fijo en p , exactamente cuando $f(p) = 0$, donde $f(x) = x^4 + 2x^2 - x - 3$:

- $g(x) = (3 + x - 2x^2)^{\frac{1}{4}}$
- $g(x) = \left(\frac{x+3-x^4}{2}\right)^{\frac{1}{2}}$

7. Demuestre que la función iterativa $g(x) = 2^{-x}$ tiene un punto fijo en $\left[\frac{1}{3}, 1\right]$

8. Use el método de punto fijo para convertir la expresión $\sqrt{3}$, con exactitud de 10^{-4}

9. ¿En qué intervalo $[a,b]$ convergerá la iteración del punto fijo con exactitud de 10^{-3} ? Para $x = \frac{5}{x^2} + 2$
10. Hallar el intervalo $[a, b]$ donde $g(x) = \sqrt{\frac{e^x}{3}}$ Tiene un punto fijo.

Use el método de Aceleración Aitken

1. $f(x) = \frac{\pi}{2}x^2 - x - 2$, $x_0 = 1.4$ (halle $g(x)$)
2. $f(x) = \cos(x) - x$, $x_0 = 0.5$ (halle $g(x)$)
3. $g(x) = \sqrt[3]{3x^2 - 4x + 1}$, $x_0 = 0.3$
4. $g(x) = e^{-x}$, $x_0 = 1$
5. $g(x) = \sqrt{3x - 2}$, $x_0 = 2$
6. $g(x) = \ln(x + 1)$, $x_0 = 0.5$
7. $g(x) = 1 - x^3$, $x_0 = 0.5$
8. $g(x) = \frac{1}{2}(x^2 - 3)$, $x_0 = 0.5$
9. $g(x) = \frac{\sin(x)+5}{x^2}$, $x_0 = 2$
10. $g(x) = x^2$, $x_0 = 0.4$, $x_0 = 0.9$, $x_0 = 1.5$
11. $g(x) = \frac{3}{2}x + \frac{1}{x^2}$, $x_0 = 0.25$

Ejercicios Newton-Raphson

1. $f(x) = (x - 1)^2$, $x_0 = 0$
2. $f(x) = x^3 - 2x - 5$, $x_0 = 1.5$
3. $f(x) = x^5 - x - 1$, $x_0 = 1$
4. Aproximar $6\sqrt{2}$ (con precisión de 8 cifras)
5. $f(x) = e^x + x^2 - 4$, $x_0 = 0.5$
6. $f(x) = x^2 - 3x - 4$, $x_0 = 8$
7. $f(x) = \ln(x) - 1$, $x_0 = 2$
8. $f(x) = x^4 - 16$, $x_0 = 2$
9. $f(x) = x^3 - 2x + 1$, $x_0 = -1.5$
10. $f(x) = e^{\{3x\}} - 4$, $x_0 = 0$
11. $f(x) = x^2 - 2x + 1$, $x_0 = 0$
12. $f(x) = xe^{-x}$, $x_0 = -1$

Polinomio Interpolante de Lagrange

El polinomio interpolante de Lagrange es una técnica matemática utilizada para aproximar funciones basándose en un conjunto de puntos dados. este método construye un polinomio que pasa exactamente por esos puntos.

$$P(x) = \sum_{i=0}^n y_i L_i(x), \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x-x_j}{x_i-x_j}$$

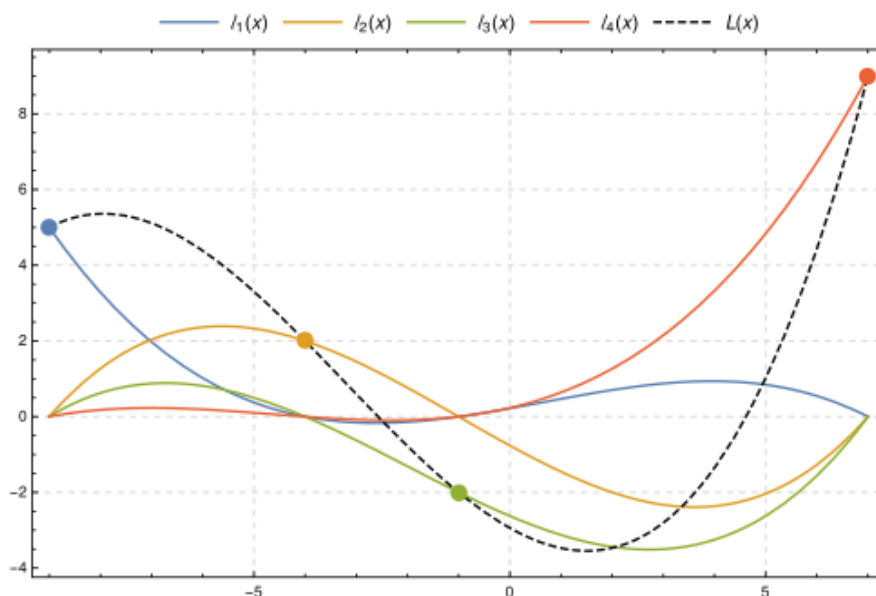


FIGURA 8

Existencia y unicidad

Siempre es posible construir un polinomio de interpolación de Lagrange para un conjunto de puntos distintos (x_i, y_i) , como $P(x)$.

$$P(x) = \sum_{i=0}^n y_i L_i(x)$$

Unicidad

Dado un conjunto $(n+1)$ puntos distintos (x_i, y_i) , existe un polinomio grado n que pasa exactamente por todos esos puntos. Esto se garantiza por el teorema fundamental de la interpolación.